Lec 5: Naming

Naveen Kumar

Assistant Professor
SVNIT, Surat

March 24, 2026

Table of Contents

1. Names and Entities
2. Flat Naming
3. Structured Naming
4. Attribute-Based Naming
   - Distributed Indexing
5. Summary

Naming: Why It Matters

The Role of Names:

- Names facilitate the sharing of resources and the unique identification of entities.
- They allow processes to refer to specific locations and provide the basis for all communication.

Entities, Access Points, and Addresses:

- To operate on an entity, a process must reach its **access point**, which is identified by an **address**.
- **Name resolution** is the process of mapping a human-friendly name to a technical address to enable entity access.

The Distributed Challenge:

- In distributed systems, the naming system itself is spread across multiple machines rather than being centralized.
- The design of this distributed implementation directly determines the overall system efficiency and scalability.

Roadmap of the Chapter

We study three major naming approaches:

1. **Flat Naming**
   - Flat naming uses unstructured, meaningless identifiers that require specialized mechanisms like Distributed Hash Tables or forwarding pointers for location-independent resolution.
2. **Structured Naming**
   - Structured naming organizes human-readable names into hierarchical name spaces, enabling systematic resolution through systems like the Domain Name System.
3. **Descriptive/Attribute-Based Naming**
   - Descriptive naming, also known as attribute-based naming, resolves entities based on characteristic descriptions which typically requires complex search operations.

Table of Contents

1. Names and Entities
2. Flat Naming
3. Structured Naming
4. Attribute-Based Naming
   - Distributed Indexing
5. Summary

Names and Entities

Definition: A **name** is a string of bits or characters used to refer to an entity.

Entities in Distributed Systems:

- Physical: hosts, disks, printers
- Logical: files, processes, users
- Communication: connections, messages

Important Observation:

- Entities provide operations through interfaces.
- Example:
  - Printer → print(), status()
  - Network connection → send(), receive(), QoS control

Key Transition: To operate on an entity → we must access it.

Access Points and Addresses

- To operate on an **entity**, we must first **access** it.
- In a distributed system:
  - An **entity** is accessed through an **access point**.
  - That access point is identified by an **address**.
- **Access Point:** The interface through which an entity is reached.
- **Address:** The name that identifies an access point. Address is **Location-dependent** in nature.
- **Example: A web server (entity)**
  - Accessed via its **network interface** (access point)
  - Identified by an **IP address + port** (address)
- **Key Observations:**
  - One entity may have **multiple access points**.
  - Access points may **change over time**.

Why Addresses Are Poor Entity Names

Distributed systems are dynamic:

- Servers may migrate to new hosts, IP addresses can be reassigned, mobile nodes may switch networks, and access points can be reused for different entities.

Problem:

- If an address is used as an entity's name, then when the entity moves the address changes and the reference becomes invalid.

Better Approach: Location-Independent Naming

$$\text{Stable Name} \Rightarrow \text{Resolve} \Rightarrow \text{Current Address}$$

Example:

- A web service has a single logical name, may run on multiple servers with different IP addresses, and DNS resolves that stable name to the current address.

Identifiers vs Addresses vs Human-Friendly Names

- As addresses can change over time, they cannot serve as stable names for entities.
- An **identifier** uniquely and permanently identifies an entity.
  - Refers to at most one entity.
  - Each entity has at most one identifier.
  - Never reused.
- Hence, **identifier equality implies the same entity**.
- A **human-friendly name** is readable and meaningful for users.
  - May not be unique.
  - Must be resolved to an address before access.
- Thus, we distinguish between:
  - **Who the entity is** → Identifier (e.g., unique server ID / hash)
  - **Where it is** → Address (e.g., 182.62.23.25:80)
  - **What we call it** → Human-friendly name (e.g., `www.google.com`)

Name Resolution

Central Question:

$$\text{Name / Identifier} \Rightarrow \text{Address}$$

- **Name resolution** is the process of mapping a name or identifier to the current address of an entity.
- A naive approach is a **centralized table** of (name, address) pairs.
- However, centralized resolution:
  - Creates a bottleneck.
  - Is not scalable.
  - Introduces a single point of failure.
- Large distributed systems therefore require decentralized and scalable resolution mechanisms.

Hierarchical Resolution and Routing

- A scalable solution uses a **hierarchical naming structure**, as in DNS.
- Example: `ftp.cs.vu.nl`
  - Query root name server: NS(.)
  - Then NS(nl)
  - Then NS(vu.nl)
  - Finally obtain the address
- **Key Insight:** Name resolution closely resembles **message routing**, as requests are forwarded step-by-step toward the responsible server.
- **Advanced Directions:**
  - Information-Centric Networking $\rightarrow$ route directly on names
  - Self-certifying names $\rightarrow$ name = hash(data)

Table of Contents

1 Names and Entities

2 Flat Naming

3 Structured Naming

4 Attribute-Based Naming
- Distributed Indexing

5 Summary

Flat Naming: What Does It Really Mean?

Recall: In structured naming (e.g., DNS),`"ftp.cs.vu.nl"` gives hints about hierarchy and location.

Flat Naming is Different:

- Identifier = random bit string.
- Example:

$$110101001010111000101011$$

- No hierarchy.
- No semantic meaning.
- No location information.

Therefore: Identifier alone does NOT help us find the entity.

Core Question: Given only identifier k, how do we locate it?

How Can Flat Names Be Resolved?

Problem:

$$\text{Given identifier } k \Rightarrow \text{Find its current address}$$

Resolution Approaches:

1. **Broadcasting / Multicasting** — Ask all (or a group) who owns k.
2. **Forwarding Pointers** — Leave a trail when the entity moves.
3. **Home-Based Tracking** — Keep a fixed home that knows the location.
4. **Distributed Hash Tables** — Use structured hashing to route to k.
5. **Hierarchical Services** — Search upward, resolve downward in domains.

Trade-offs: Simplicity vs Scalability, Latency vs Robustness, Global search vs Locality.

Broadcasting

Principle:

- A node broadcasts the identifier k to every machine in the network.
- The machine that owns k replies with its current access address.

Applicable Environments:

- Works efficiently in local-area networks where broadcast is hardware-supported.
- Also feasible in small wireless LANs with a single broadcast domain.

Limitations:

- Every broadcast consumes network bandwidth, even for nodes not involved.
- All hosts must process the request, causing unnecessary interruption.
- The approach does not scale to large or wide-area networks.

Example: ARP with Address Fields

Goal: Resolve logical IP to physical MAC address. IP $\longrightarrow$ MAC **Scenario:** Host A:$IP = 192.168.1.10, \quad MAC = AA:AA:AA:AA:AA:AA$
Wants to send data to: $IP = 192.168.1.20$ **1. ARP Request (Broadcast)**

- Ethernet Destination MAC = FF:FF:FF:FF:FF:FF (Broadcast)
- Ethernet Source MAC = AA:AA:AA:AA:AA:AA
- ARP Sender IP = 192.168.1.10
- ARP Target IP = 192.168.1.20

Message: "Who has 192.168.1.20? Tell 192.168.1.10." **2. ARP Reply (Unicast)**
Host B:$IP = 192.168.1.20, \quad MAC = BB:BB:BB:BB:BB:BB$

- Ethernet Destination MAC = AA:AA:AA:AA:AA:AA
- Ethernet Source MAC = BB:BB:BB:BB:BB:BB
- ARP Sender IP = 192.168.1.20
- ARP Target IP = 192.168.1.10

Hooo! I know the MAC address of the host "$IP = 192.168.1.20$" which is $MAC = BB:BB:BB:BB:BB:BB$

Key Insight: ARP uses broadcast for discovery and unicast for response.

Forwarding Pointers

Motivation: An entity may move frequently, but clients still use the same identifier.

Basic Idea: When the entity moves from location A to B:

- A stores a reference (forwarding pointer) to B.
- Future requests sent to A are redirected to B.

If the entity moves again (B $\rightarrow$ C):

- B stores a pointer to C.
- A $\rightarrow$ B $\rightarrow$ C forms a chain.

Lookup Process:

$$\text{Client} \rightarrow A \rightarrow B \rightarrow \cdots \rightarrow \text{Current Location}$$

Main Problems:

- Long chains increase lookup latency.
- A broken link makes the entity unreachable.
- Each intermediate location must maintain state.

SSP Chains: Client/Server Stub Model

Purpose: Implement forwarding pointers transparently for remote objects.

Components:

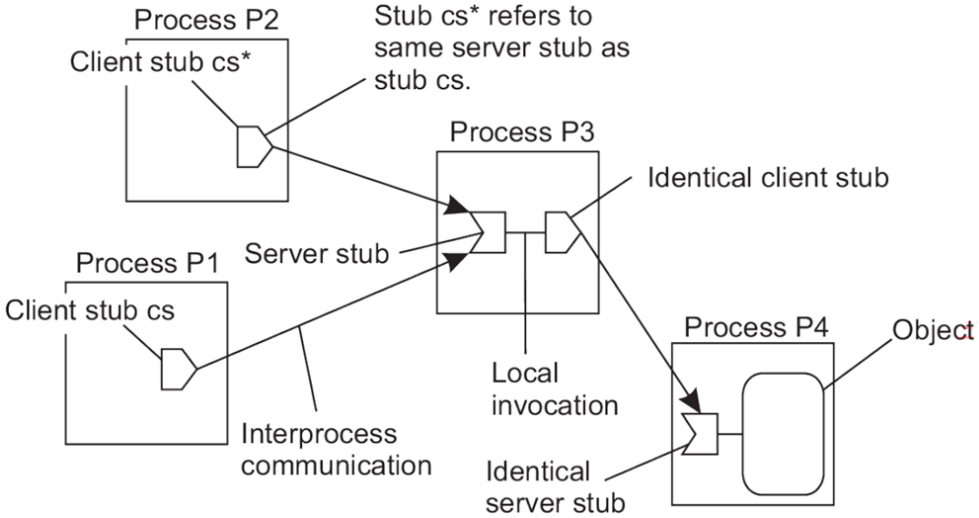
- **Client Stub** Interface visible to the client process.
- **Server Stub** Holds either:
  - A local reference to the object, or
  - A reference to another client stub.

Migration Behavior:

- Object moves → new stubs installed.
- Client continues invoking same stub.
- Migration remains transparent.

Invocation Flow:

Client → Stub Chain → Object



Forwarding implemented using (client stub, server stub) pairs.

Shortcut Optimization in SSP Chains

Problem: Forwarding chains may grow long, increasing latency.

Optimization Mechanism:

- Each invocation carries the initiating client-stub ID.
- When the request reaches the current object location, the response is sent directly back to that stub.
- The response piggybacks the object's current location.
- The client stub updates its server reference (shortcut).

Effect: Chain length reduces over time
Trade-off: Direct response is faster, but updating all intermediate stubs may require reverse-path traversal.

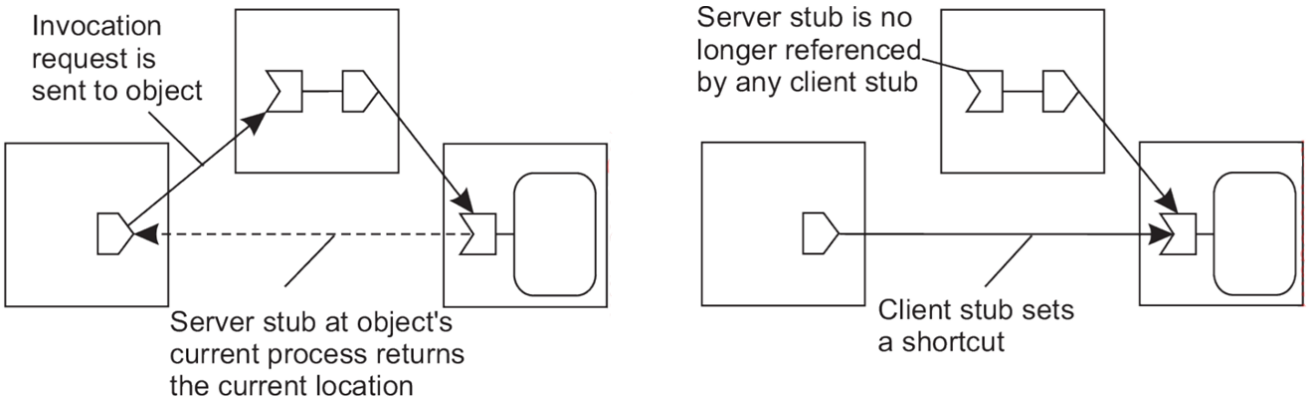


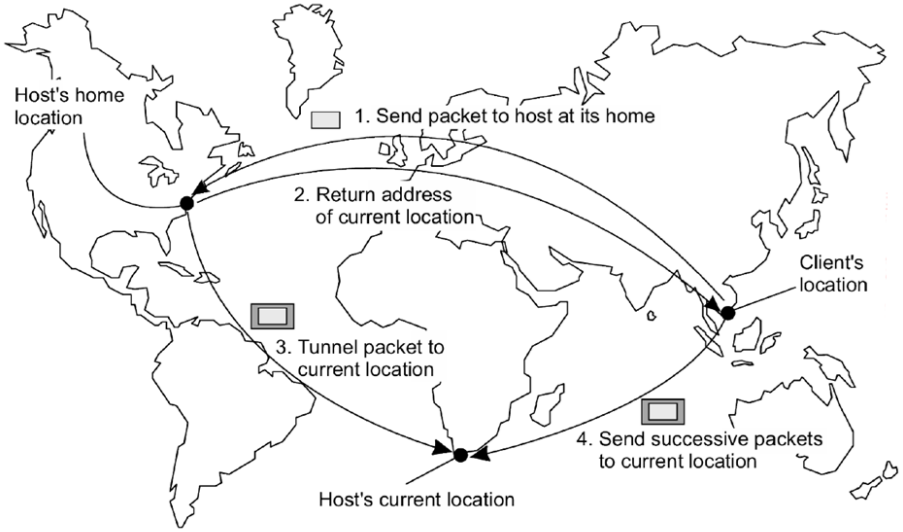
Figure 1: Redirecting a forwarding pointer by storing a shortcut in a client stub.

Home-Based Location Tracking

- Each entity is assigned a fixed home location that stores its current address.
- A client first contacts the home, which either returns the entity's current address or forwards the request.
- The identifier remains stable because location changes are handled at the home.
- This approach is simple and provides strong location transparency to applications.
- However, indirect routing increases communication latency and the home may become a bottleneck or single point of failure.

Mobile IP Example

In Mobile IP, a host keeps a permanent IP address that serves as its stable identifier.
When it moves, it obtains a temporary care-of address representing its current location.
The home agent maintains the mapping and tunnels packets from the permanent IP to the care-of address.
This ensures location transparency, but may introduce additional routing latency.



The principle of Mobile IP.

Distributed Hash Tables

In a Distributed Hash Table (DHT), both nodes and keys are assigned identifiers from the same m-bit circular space.
A key k is stored at its successor node, defined as

$$succ(k) = \min\{id \mid id \geq k\}.$$

The system routes lookup requests efficiently so that resolving a key requires only $O(\log N)$ hops.

Chord Lookup Example

Finger table enables exponential shortcuts.
Lookup requires:

$$O(\log N) \text{ hops}$$

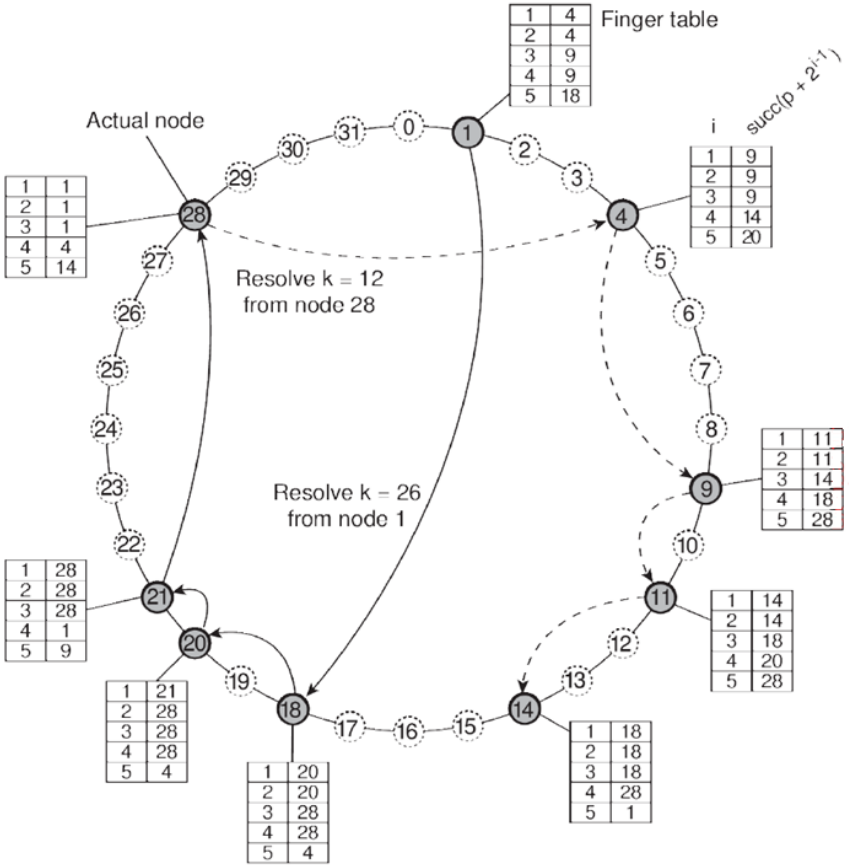


Figure 2: Resolving key 26 from node 1 and key 12 from node 28 in a Chord system.

Hierarchical Location Service: Structure

- **Domain Hierarchy:** The network is organized into nested domains. A *leaf domain* (e.g., a LAN) is the smallest unit, while the *root domain* spans the entire network.
- **Directory Nodes:** Each domain D has an associated directory node $dir(D)$ that tracks entities within its boundaries.
- **Locality Assumption:** The design assumes that message transfer within a smaller domain is faster than in a larger one.
- **Pointer Chains:** Higher-level nodes don't store addresses; they store **pointers** to the next lower-level directory node.

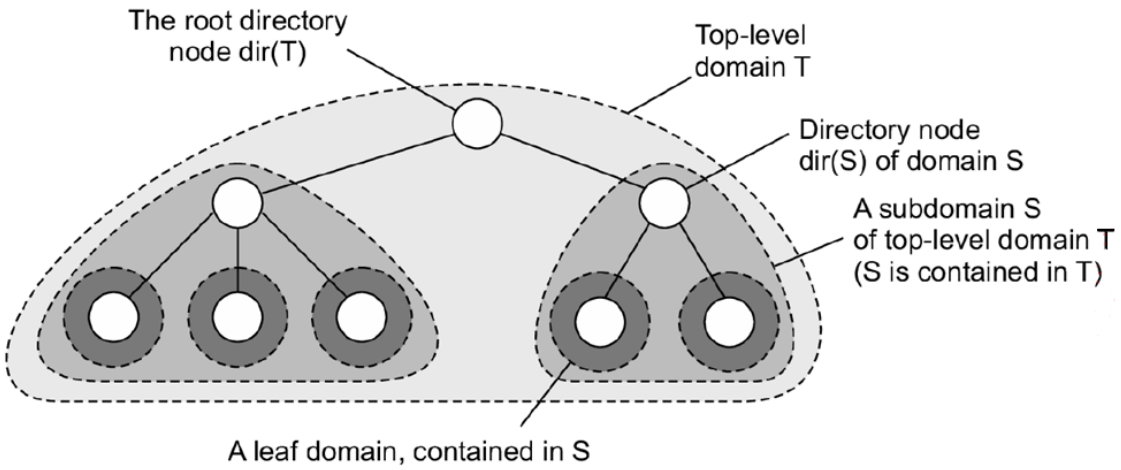


Figure 5.6: Hierarchical organization of domains and directory nodes.

Entity with Multiple Addresses

- **Replication:** When an entity E exists in multiple leaf domains (e.g., D_1 and D_2), it possesses multiple valid addresses.
- **Smallest Common Ancestor:** The directory node of the smallest domain containing all replicas stores multiple pointers—one for each path leading to a replica.
- **Global Reachability:** Every entity has a record in the root node, ensuring that any client can eventually find a path to the entity.

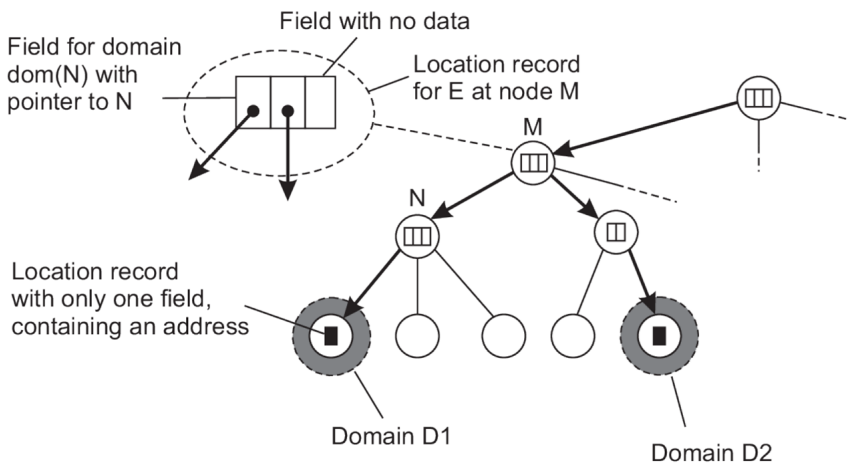


Figure 5.7: Storing location records for an entity replicated across leaf domains.

Hierarchical Lookup: The "Expanding Ring"

- **Phase 1: Upward Search**
  - Starts at the client's local leaf node.
  - If no record is found, it forwards to the parent. This explores a "gradually increasing ring" around the client to exploit locality.
- **Phase 2: Downward Resolution**
  - Once a record is found at node M, the request follows pointers down to the leaf node holding the actual address.

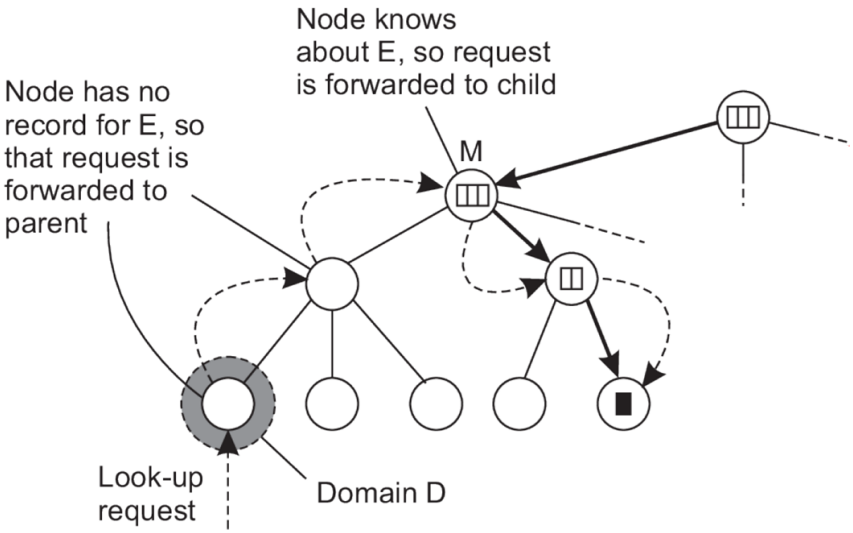


Figure 3: Lookup proceeds upward until a record is found, then downward to the leaf.

Insert Operation: Registration Strategies

Step 1: Upward Propagation

- Forwarded from leaf node until a node already storing a record for E is reached.

Step 2: Pointer Creation

- **Top-Down:** Pointers created along the reverse path.
- **Bottom-Up (Advantage):** Records created *before* forwarding. This allows local lookups even if parent nodes are temporarily unreachable.

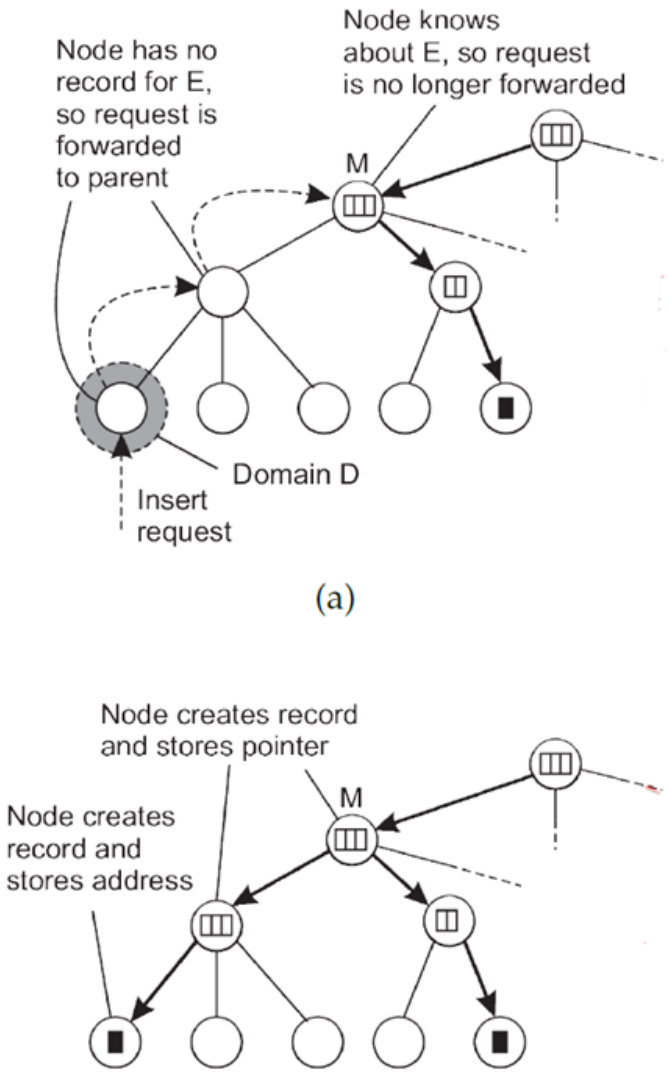


Figure 5.9: (a) Upward request. (b) Pointer chain construction.

Delete Operation

- **Initiation:** Triggered when an address for entity E is removed from a leaf domain D.
- **Recursive Cleanup:**
  - The leaf node $dir(D)$ removes the address.
  - If the record for E becomes empty, the leaf node is deleted, and the parent is notified to remove its pointer.
  - This continues upward until a directory node is reached that still contains other valid pointers for E.

Result: The hierarchy remains clean and only maintains paths to existing replicas.

Scalability: The Logical Root Bottleneck

- **The Problem:** In a theoretical tree, the root node must store a record for *every* entity in the world.
- **The Bottleneck:** If implemented as a single machine, the root would be:
  - A performance bottleneck (too much traffic).
  - A single point of failure (if the root crashes, the system fails).
- **The Solution:** Decouple the **Logical Hierarchy** from the **Physical Infrastructure**.
  - **Logical:** The "Map" of how to find things (the tree).
  - **Physical:** The actual server racks (Hosts $H_1, \ldots, H_N$) that store the data.

Core Principle

The "Root" is not a machine; it is a **role** spread across many machines.

Physical Implementation of the Hierarchy

How it works:

- **Entity IDs:** Every entity is assigned a random m-bit identifier (similar to Chord/DHTs).
- **Location Servers (LS):** Each physical host runs multiple "servers." One host might be a leaf for Entity A but the *root* for Entity B.
- **Shared Responsibility:**
  - No single host is "The Root."
  - The load of the top-level domains is partitioned across all available physical hosts.

Result:

- *Geographical Scaling:* Handled by the hierarchy (lookups stay local).
- *Size Scaling:* Handled by distribution (adding hosts adds capacity).

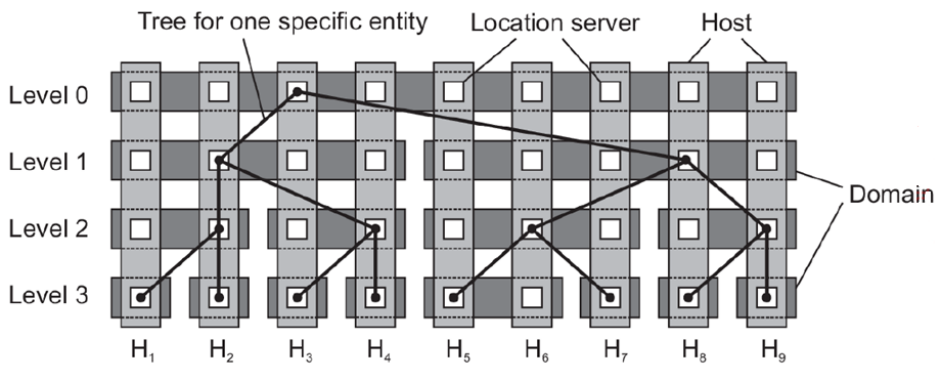


Figure 4: Mapping logical tree levels to physical hosts $H_1 \ldots H_9$.

Table of Contents

1 Names and Entities

2 Flat Naming

3 Structured Naming

4 Attribute-Based Naming
- Distributed Indexing

5 Summary

Structured Naming in Distributed Systems

- **From Flat to Structured Names:** Flat names (e.g., random identifiers, hashes) are efficient for machines, but difficult for humans to interpret.
- **Structured Names:** Composed of meaningful labels organized hierarchically (e.g., file paths `/home/user/doc.txt`, URLs, domain names).
- **Core System Objective:** Map a human-readable name to the correct entity (process, file, service, or resource) in the system.
- **Key Questions We Will Study:**
  - How is a name space structured?
  - How is a name resolved?
  - How is the name space distributed?
  - How do systems like DNS and NFS implement this?

Name Spaces as Labeled Directed Graphs

- **Abstract Model:** A name space is modeled as a **labeled, directed graph**.
- **Nodes Represent:**
  - **Leaf Nodes:** Actual entities (files, services, processes). No outgoing edges. Store identifiers, addresses, or state.
  - **Directory Nodes:** Contain a **Directory Table** mapping:

    label $\rightarrow$ child node identifier
- **Root Node:** Single entry point of the name space.
- **Important:** Structure is a **general graph** (not always a strict tree).

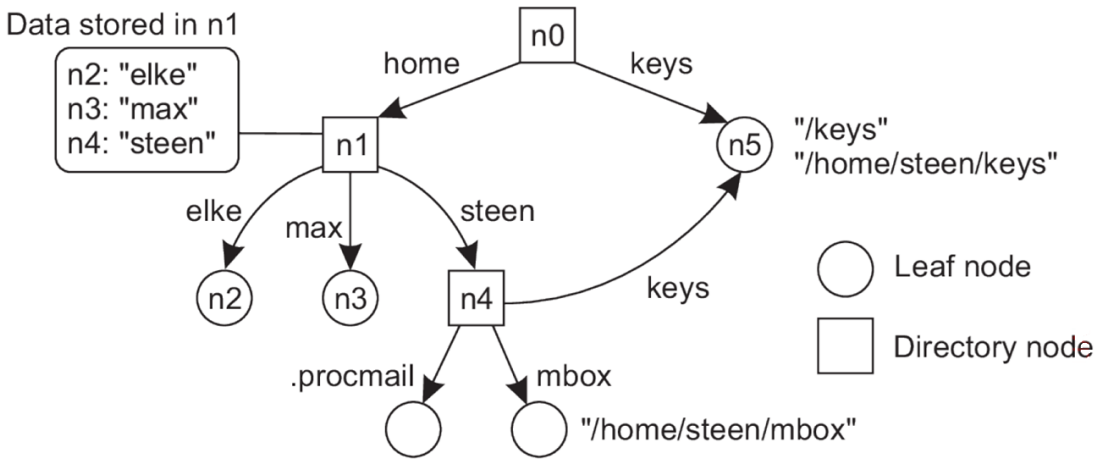


Fig 5.11: General naming graph

Path Names and String Representations

- **Path Name (Graph View):** A sequence of labels

$$N = [l_1, l_2, \ldots, l_n]$$

  that guides traversal in the naming graph.
- **String Form (User View):** Written as a readable string, e.g., `/home/user/report.pdf`
- **Absolute vs Relative:** Absolute paths start at the root; relative paths start from the current directory.
- **Structure: Tree** (single parent) or **DAG** (multiple parents, no cycles).
- **Scope: Global names** have the same meaning everywhere; **local names** depend on context.

Unix File System: Concrete Implementation of Naming

- **Inode** Each file or directory is represented by an **inode** (internal identifier).
- **Directory = Mapping Table**

  name $\rightarrow$ inode number

  Names are stored in directories, not inside inodes.
- **Disk Organization:** Boot block $\rightarrow$ Superblock $\rightarrow$ Inode table $\rightarrow$ Data blocks
- **Root Directory:** Fixed inode number (starting point of resolution).

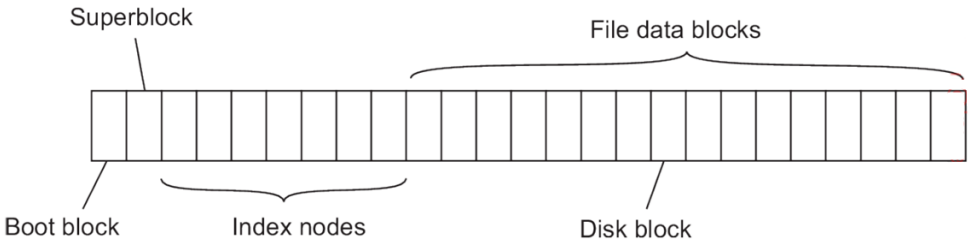


Fig 5.12: Unix file system layout on disk.

Name Resolution: How a Path Becomes a File

Step-by-Step Resolution:

- Given a path $[l_1, l_2, \ldots, l_n]$, process one label at a time.
- At each directory:

$$\text{name} \rightarrow \text{inode number}$$

- Follow the inode to:
  - another directory (continue), or
  - a file (stop).

Closure Mechanism:

- The system must know where resolution starts.
- In Unix: the root inode location is fixed.

Hard Links vs Symbolic Links

Hard Link

- Adds another directory entry:

$$\text{name}_1,\ \text{name}_2 \rightarrow \text{same inode}$$

- No new inode, no data duplication.
- File removed only when **link count = 0**.
- Cannot cross file systems.

Symbolic Link

- Creates a new inode storing a **path string**.
- During access: read stored path → restart resolution.
- Can cross file systems.
- May become **dangling**.

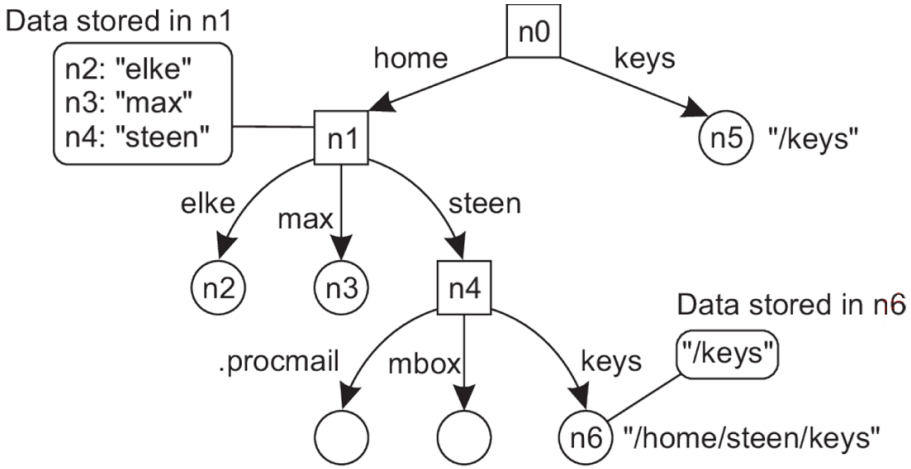


Fig 5.13: Symbolic link example.

Mounting Across Networks (Distributed Name Spaces)

Mount Point

- A local directory that acts as an entry point to a **remote name space**.
- Example (Fig 5.14): `"/remote/vu"` refers to a directory on Machine B.

How It Works

- Local system stores:
  - Server address
  - Access protocol (e.g., NFS)
  - Remote directory identifier
- When resolution reaches the mount point, requests are forwarded over the network.

Result:

- Remote directory tree is merged into the local view.
- Appears as a single unified file system.

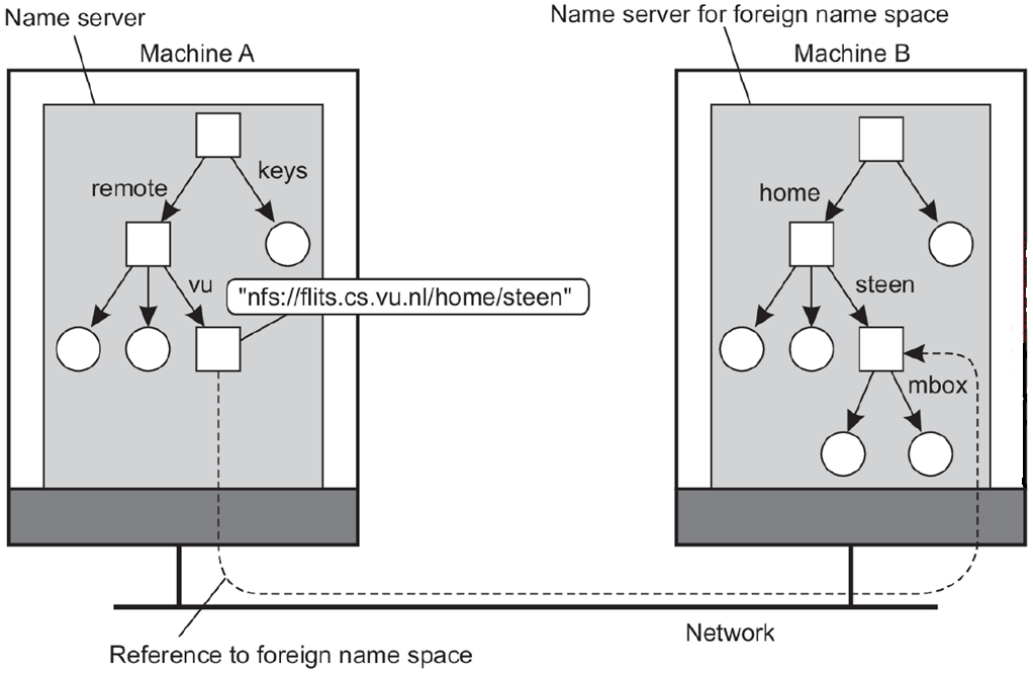


Fig 5.14: Mounting remote name spaces via protocol.

Name Space Distribution in DNS

Logical Layers (Fig 5.15)

1. **Global Layer**
   - Root and TLDs (.com, .edu, .org)
   - Very stable
   - Highly replicated worldwide
2. **Administrative Layer**
   - Organization domains (e.g., vu.nl)
   - Managed by institutions
3. **Managerial Layer**
   - Individual hosts/services
   - High update frequency

Key Idea: Different layers → different stability and management.

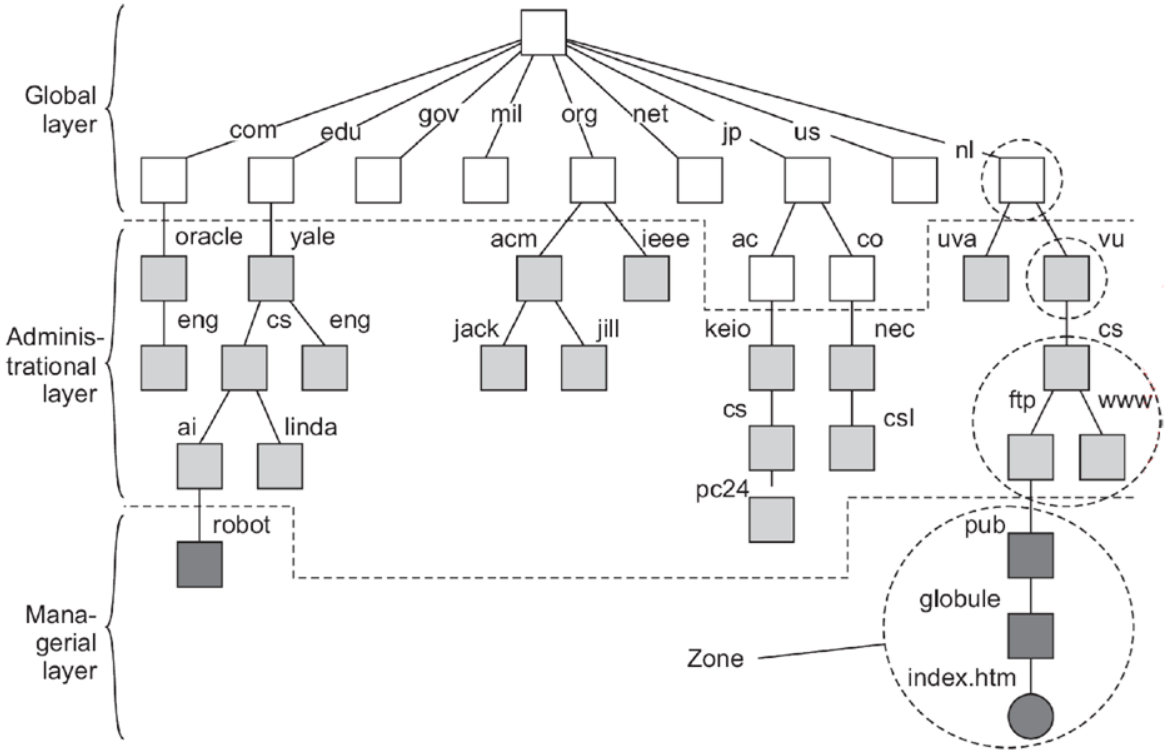


Fig 5.15: DNS name space partitioned into layers and zones.

Comparison of Name Space Layers

Issue	Global	Administrational	Managerial
Geographical scale	Worldwide	Organization	Department
Number of nodes	Few	Many	Vast numbers
Responsiveness to lookups	Seconds	Milliseconds	Immediate
Update propagation	Lazy	Immediate	Immediate
Number of replicas	Many	None or few	None
Client-side caching	Yes	Yes	Sometimes

- **Global Layer:** worldwide scale, high replication, lazy updates.
- **Managerial Layer:** local scale, immediate updates, rarely replicated.

Iterative Name Resolution (DNS)

Key Idea: Client Drives Resolution

- The client's resolver contacts each server step by step.
- Servers do **not** contact other servers on behalf of the client.

How It Works (Fig 5.17)

- Query root for `nl.vu.cs.ftp`
- Root returns address of `.nl` server
- `.nl` returns address of `vu` server
- Continue until final host (`ftp`)

Why Iterative?

- Each server resolves only its level.
- Reduces workload on root/TLD servers.

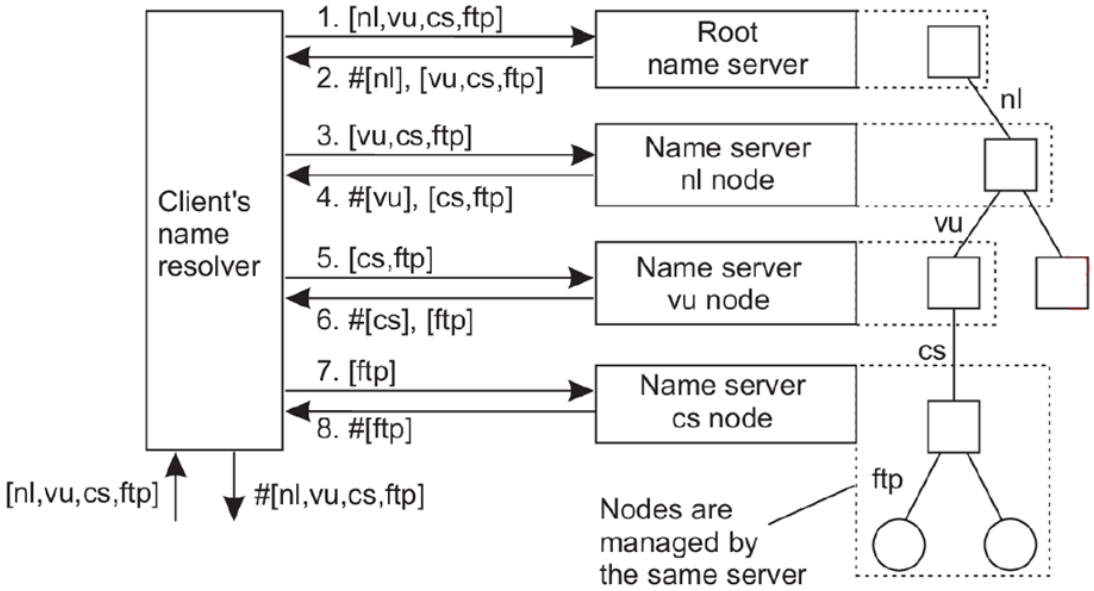


Fig 5.17: Step-by-step iterative resolution.

Recursive Name Resolution (DNS)

Key Idea: Server Drives Resolution

- Client sends one request.
- The first server resolves the name **on behalf of the client**.
- Servers contact other servers down the hierarchy.

How It Works (Fig 5.18)

- Client → Root
- Root → .nl → vu → cs
- Final answer travels back up the chain to the client.

Characteristics

- Higher load on upper-level servers.
- Fewer round-trips for the client.
- Enables effective caching at recursive servers.

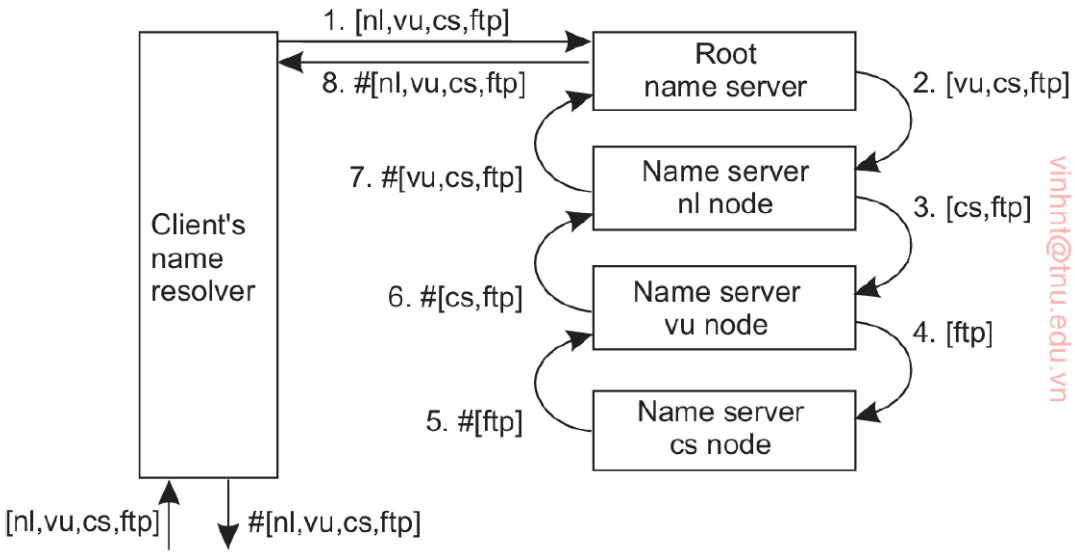


Fig 5.18: Server-driven recursive resolution.

vinhnt@tnu.edu.vn

Caching in Recursive Resolution

Key Idea: Recursive servers cache intermediate results.

- While resolving `nl.vu.cs.ftp`, servers learn addresses of: `.nl`, `vu`, `cs`.
- Results are cached (with TTL).
- Later lookups (e.g., `nl.vu.cs.mail`) can skip upper levels.
- Greatly reduces load on root/TLD servers.

Server for node	Should resolve	Looks up	Passes to child	Receives and caches	Returns to requester
cs	[ftp]	#[ftp]	—	—	#[ftp]
vu	[cs, ftp]	#[cs]	[ftp]	#[ftp]	#[cs] #[cs, ftp]
nl	[vu, cs, ftp]	#[vu]	[cs, ftp]	#[cs] #[cs, ftp]	#[vu] #[vu, cs] #[vu, cs, ftp]
root	[nl, vu, cs, ftp]	#[nl]	[vu, cs, ftp]	#[vu] #[vu, cs] #[vu, cs, ftp]	#[nl] #[nl, vu] #[nl, vu, cs] #[nl, vu, cs, ftp]

Fig 5.19: Intermediate results cached during recursion.

Communication Costs: Iterative vs Recursive

Iterative

- Client contacts each server.
- Multiple long-distance round trips.
- Lower server workload.

Recursive

- One client request.
- Servers contact each other.
- Fewer client round trips.

Trade-off: Client load vs server load.

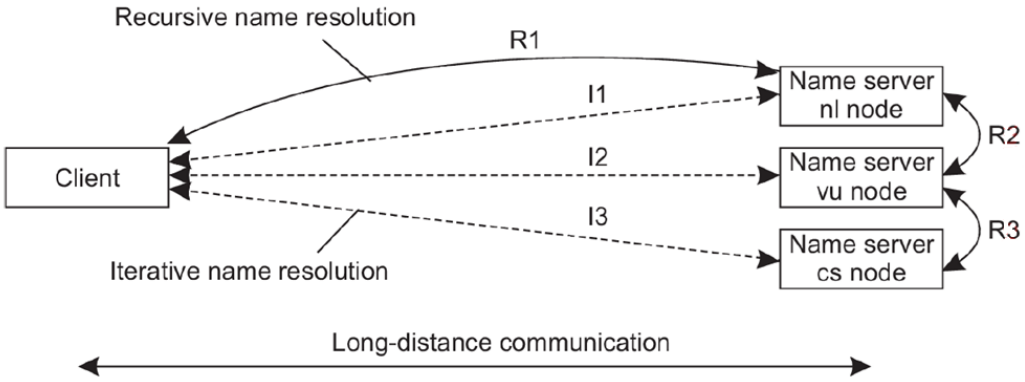


Fig 5.20: Communication comparison.

DNS Structure and Resource Records

Every entry in the DNS database is called a **Resource Record (RR)**.

- **A (Address):** The most common record. It maps a hostname to an IPv4 address.
- **NS (Name Server):** Identifies which server is the *authoritative name server* for a specific domain zone.
- **CNAME (Canonical Name):** An alias record that maps one domain name to another domain name.
- **SOA (Start of Authority):** The header of the DNS zone file. It contains administrative information such as the primary name server and update policies.
- **MX (Mail Exchange):** Specifies the mail servers responsible for receiving email messages for the domain.

Example DNS Zone Files

Type	Refers to	Description
SOA	Zone	Holds info on the represented zone
A	Host	IP addr. of host this node represents
MX	Domain	Mail server to handle mail for this node
SRV	Domain	Server handling a specific service
NS	Zone	Name server for the represented zone
CNAME	Node	Symbolic link
PTR	Host	Canonical name of a host
HINFO	Host	Info on this host
TXT	Any kind	Any info considered useful

Fig 5.21: Important types of resource records.

- Databases are implemented as a collection of zone files.
- Note the use of **CNAME** (aliases) and multiple **NS** records for availability.

Name	Record type	Record value
cs.vu.nl.	SOA	star.cs.vu.nl. hostmaster.cs.vu.nl. 2005092900 7200 3600 2419200 3600
cs.vu.nl.	TXT	"VU University - Computer Science"
cs.vu.nl.	MX	1 mail.few.vu.nl.
cs.vu.nl.	NS	ns.vu.nl.
cs.vu.nl.	NS	top.cs.vu.nl.
cs.vu.nl.	NS	solo.cs.vu.nl.
cs.vu.nl.	NS	star.cs.vu.nl.
star.cs.vu.nl.	A	130.37.24.6
star.cs.vu.nl.	A	192.31.231.42
star.cs.vu.nl.	MX	1 star.cs.vu.nl.
star.cs.vu.nl.	MX	666 zephyr.cs.vu.nl.
star.cs.vu.nl.	HINFO	"Sun" "Unix"
zephyr.cs.vu.nl.	A	130.37.20.10
zephyr.cs.vu.nl.	MX	1 zephyr.cs.vu.nl.
zephyr.cs.vu.nl.	MX	2 tornado.cs.vu.nl.
zephyr.cs.vu.nl.	HINFO	"Sun" "Unix"
ftp.cs.vu.nl.	CNAME	soling.cs.vu.nl.
www.cs.vu.nl.	CNAME	soling.cs.vu.nl.
soling.cs.vu.nl.	A	130.37.20.20
soling.cs.vu.nl.	MX	1 soling.cs.vu.nl.
soling.cs.vu.nl.	MX	666 zephyr.cs.vu.nl.
soling.cs.vu.nl.	HINFO	"Sun" "Unix"
vucs-das1.cs.vu.nl.	PTR	0.198.37.130.in-addr.arpa.
vucs-das1.cs.vu.nl.	A	130.37.198.0
inkt.cs.vu.nl.	HINFO	"OCE" "Proprietary"
inkt.cs.vu.nl.	A	192.168.4.3
pen.cs.vu.nl.	HINFO	"OCE" "Proprietary"
pen.cs.vu.nl.	A	192.168.4.2
localhost.cs.vu.nl.	A	127.0.0.1

Delegation Between Zones

- A parent zone delegates responsibility by storing **NS records** for the child zone.
- "Glue records" provide the IP address of the child zone's name server to prevent circular dependencies.

Name	Record type	Record value
cs.vu.nl.	NS	solo.cs.vu.nl.
cs.vu.nl.	NS	star.cs.vu.nl.
cs.vu.nl.	NS	ns.vu.nl.
cs.vu.nl.	NS	top.cs.vu.nl.
ns.vu.nl.	A	130.37.129.4
top.cs.vu.nl.	A	130.37.20.4
solo.cs.vu.nl.	A	130.37.20.5
star.cs.vu.nl.	A	130.37.24.6
star.cs.vu.nl.	A	192.31.231.42

Fig 5.23: Delegation for the `cs.vu.nl` domain.

NFS Naming Model

- Clients build their own name spaces by mounting remote directories.
- **Non-Global Names:** `/remote/vu/mbox` on Client A might be `/work/me/mbox` on Client B.
- Makes cross-user sharing difficult.

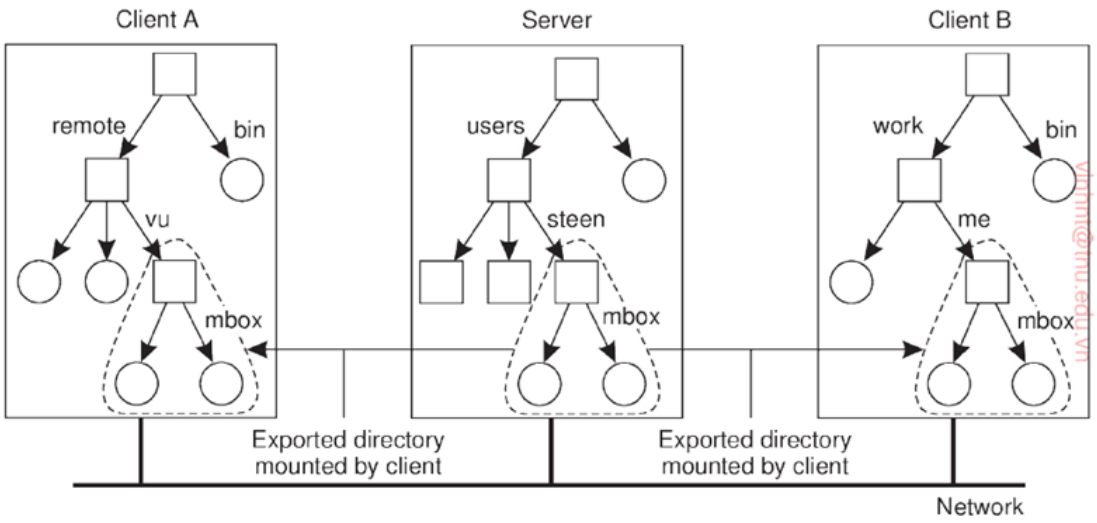


Fig 5.24: Mounting (part of) a remote file system.

Nested Mounting in NFS

- Servers cannot export directories they have mounted from another server.
- Clients must mount nested directories directly from the source server.
- Ensures iterative resolution remains simple for the server.

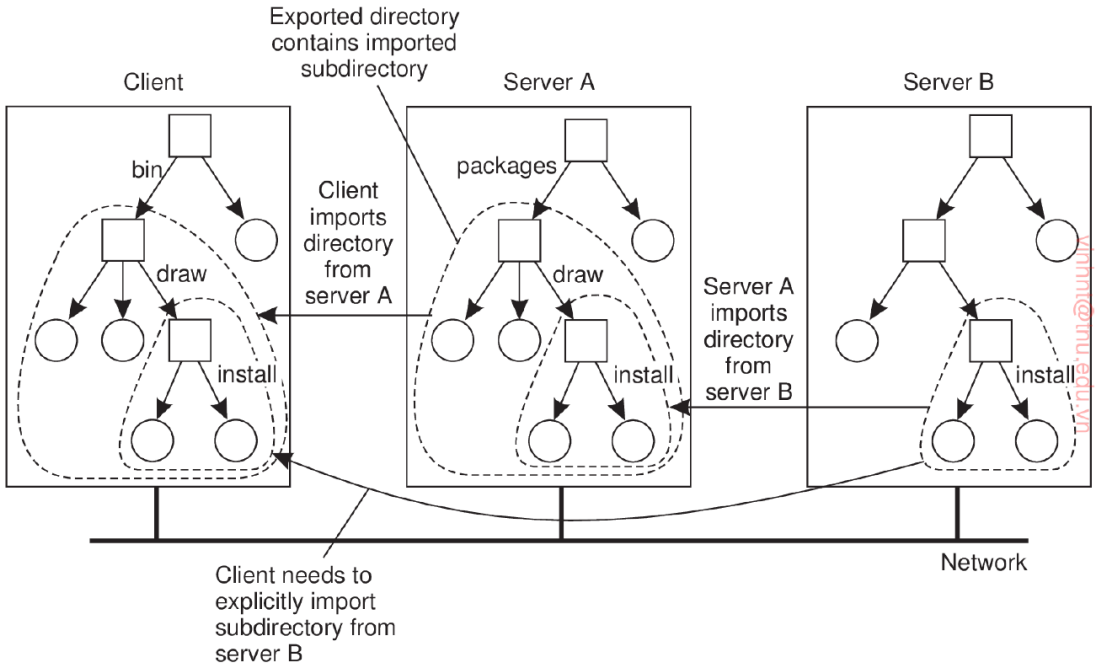


Fig 5.25: Mounting nested directories from multiple servers.

File Handles and NFSv4 Improvements

File Handles

- In NFS, clients access remote files on a server.
- Repeatedly resolving full path names is inefficient.
- The server therefore provides a **file handle** — a compact identifier for a file or directory.
- Clients use this identifier in later operations instead of sending the full path again.

NFSv4 Improvements

- The server handles directory traversal and mount points.
- Clients start from the root using `putrootfh`.
- Mounting and naming are integrated, simplifying file lookup.

NFS Automounting

- **Strategy:** Mount remote systems only when they are first accessed to save boot time and resources.
- **Process:** An automounter process intercepts lookups and performs the mount on-demand.

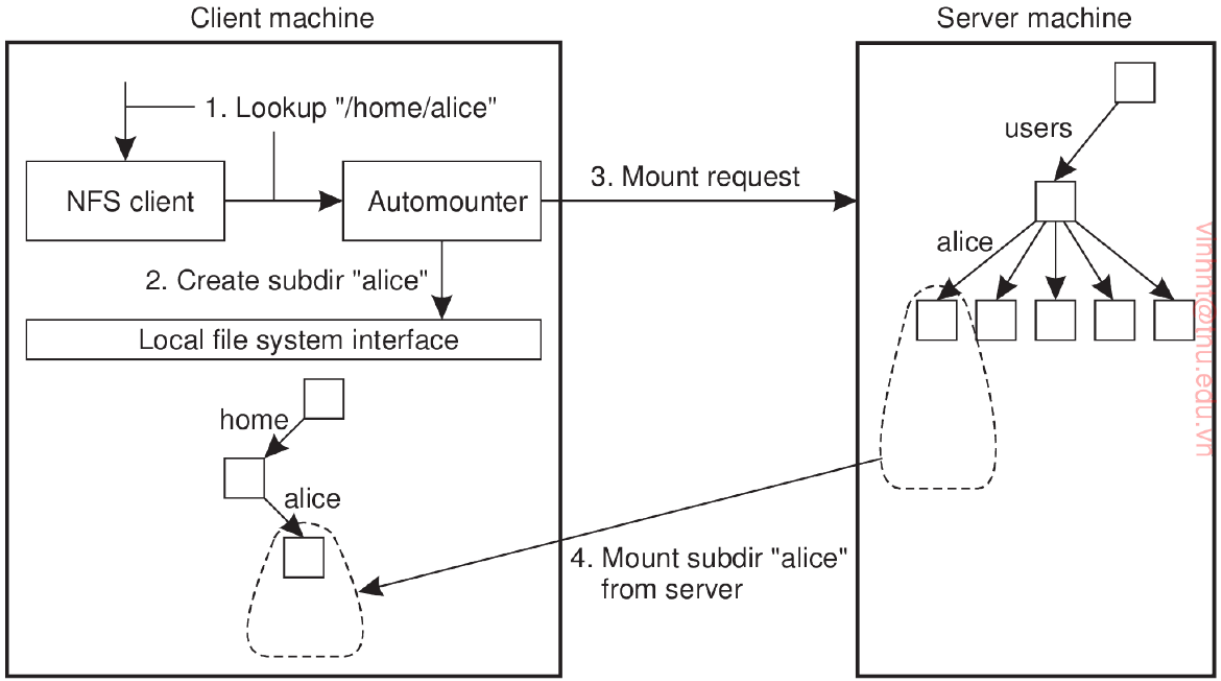


Fig 5.26: A simple automounter for NFS.

Optimized Automounting with Symbolic Links

- To reduce this overhead of using automounter every time, the system mounts the remote file system in a **hidden directory**.
- A **symbolic link** is then created that points to the mounted location to bypass the automounter for further request.
- This reduces overhead and improves performance for subsequent reads and writes.

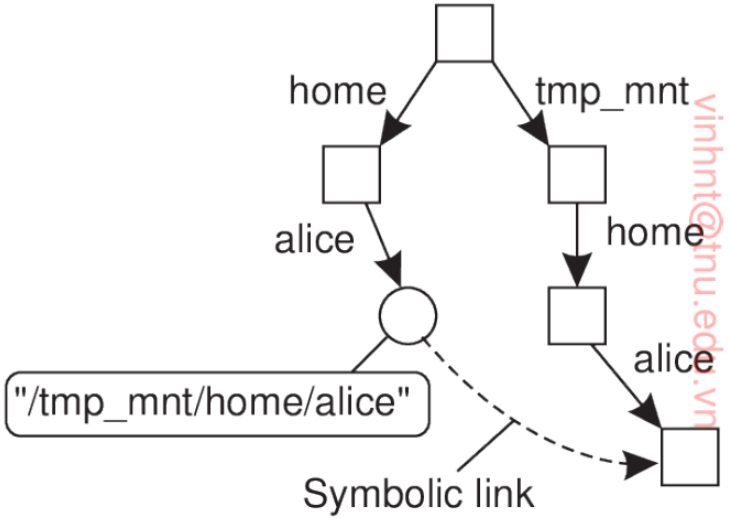


Fig. 5.27: Automounting using symbolic links.

vinhnt@tnu.edu.vn

Table of Contents

1. Names and Entities
2. Flat Naming
3. Structured Naming
4. Attribute-Based Naming
   - Distributed Indexing
5. Summary

Why Attribute-Based Naming?

- **Flat Naming:**
  - Uses unique identifiers (e.g., file ID)
  - No search capability
- **Structured Naming:**
  - Uses hierarchical paths (e.g., directories)
  - Requires knowing the exact location
- **Attribute-Based Naming:**
  - Search using **descriptions**
  - Based on attribute-value pairs

Example Query:

$$(a_1 = v_1) \wedge (a_2 = v_2)$$

Core Idea

- Each entity is represented as:

$$\{(a_1, v_1), (a_2, v_2), \ldots, (a_n, v_n)\}$$

- **Search Process:**
  - Match entities satisfying given constraints
- **Key Difference:**
  - Naming systems $\rightarrow$ Exact lookup
  - Directory services $\rightarrow$ Content-based search
- **Important:**
  - Search is computationally harder than lookup

Design Challenges

- Designing effective attribute-based systems involves:
  - Defining consistent attribute sets
  - Ensuring uniform value representation
  - Handling ambiguity in descriptions
  - Avoiding expensive distributed searches
- No direct path exists (unlike structured naming)
- **Main concern: Scalability**

Resource Description Framework (RDF)

- Data is represented as triplets:

$$(\text{Subject}, \text{Predicate}, \text{Object})$$

- Example:

$$(\text{Person}, \text{name}, \text{Alice})$$

- Each component can be a resource (e.g., URL)
- Forms a **graph structure**
- Enables machine-readable metadata

LDAP Directory Model

- LDAP: A hierarchical directory service
- Each entry consists of:
  - Attribute-value pairs
- Attributes can be:
  - Single-valued
  - Multi-valued

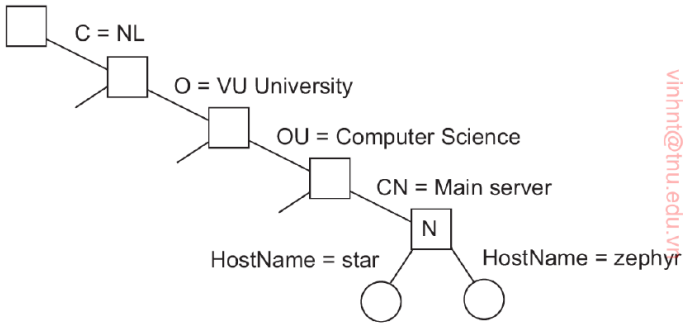
Attribute	Abbr.	Value
Country	C	NL
Locality	L	Amsterdam
Organization	O	VU University
OrganizationalUnit	OU	Computer Science
CommonName	CN	Main server
Mail_Servers	–	137.37.20.3, 130.37.24.6, 137.37.20.10
FTP_Server	–	130.37.20.20
WWW_Server	–	130.37.20.20

Directory Information Tree (DIT)

- Entries are organized hierarchically
- Each node identified by:
  - Relative Distinguished Name (RDN)
- Example:

  $/C = NL/O = VU\ University/OU = Computer\ Science$

- Forms a **Directory Information Tree (DIT)**



(a)

Attribute	Value	Attribute	Value
Locality	Amsterdam	Locality	Amsterdam
Organization	VUUniversity	Organization	VUUniversity
OrganizationalUnit	ComputerScience	OrganizationalUnit	ComputerScience
CommonName	Mainserver	CommonName	Mainserver
HostName	star	HostName	zephyr
HostAddress	192.31.231.42	HostAddress	137.37.20.10

(b)

vinhnt@tnu.edu.vn

LDAP Operations

- **Supported Operations:**
  - read $\rightarrow$ Retrieve a single entry
  - list $\rightarrow$ Retrieve child entries
  - search $\rightarrow$ Attribute-based filtering
- Example Query:

$$(C = NL)(O = VU\ University)(OU = *)(CN = Main\ server)$$

- Searches may span multiple nodes

Distributed LDAP

- Large directories are partitioned across:
  - Directory Service Agents (DSA)
- Clients interact via:
  - Directory User Agents (DUA)
- Active Directory introduces:
  - Global catalog for indexing
- Trade-off:
  - Hierarchical control vs search efficiency

Table of Contents

1. Names and Entities
2. Flat Naming
3. Structured Naming
4. Attribute-Based Naming
   - Distributed Indexing
5. Summary

Problem: Distributed Search

- Data is distributed across multiple nodes
- Queries use attribute-value constraints:

$$F(a_1, v_1) \wedge F(a_2, v_2)$$

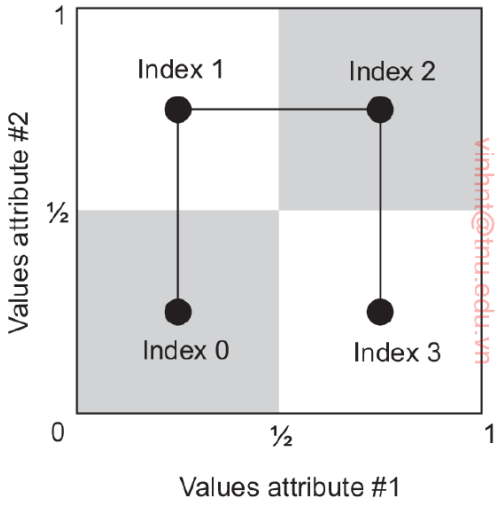
- Goal:

$$L(a_1, v_1) \cap L(a_2, v_2)$$

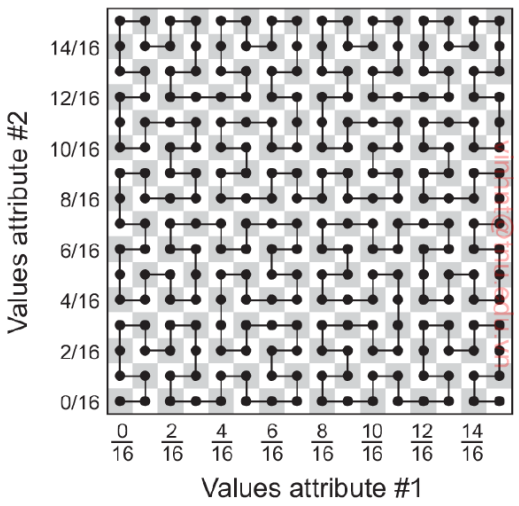
- **Challenges:**
  - High communication cost
  - Large intermediate results
  - Poor support for range queries

Space-Filling Curves (SFC)

- Treat attributes as coordinates in N-dimensional space
- Map multi-dimensional space to 1D using:
  - Hilbert curve
- Preserves locality:
  - Nearby points remain close in 1D
- **Advantage:**
  - Fewer nodes need to be contacted



(a)



(b)

Example: Hilbert Mapping

- Attributes:

$$\text{CPU} = v_1, \quad \text{RAM} = v_2$$

- Each resource mapped to 1D index
- Example:

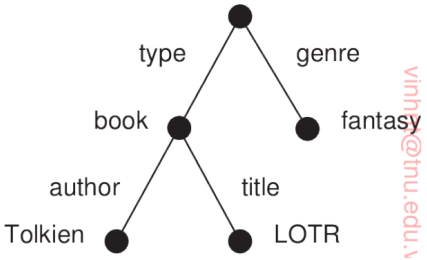
Server	(v_1, v_2)	Index
S1	(0.2, 0.4)	5
S2	(0.4, 0.8)	9
S3	(0.6, 0.8)	10
S4	(0.8, 0.9)	15

- Query $\rightarrow$ index range [9,10]
- Result $\rightarrow$ S2, S3

INS / Twine Approach

- Uses Attribute-Value Trees (AVTrees)
- Each path is hashed into a key:
  - Example: type $\rightarrow$ book $\rightarrow$ author $\rightarrow$ Tolkien
- Supports partial queries
- Improves flexibility in search

```
description {
    type = book
    description {
        author = Tolkien
        title = LOTR
    }
    genre = fantasy
}
```



vinhny@tnu.edu.v

SWORD Technique

- Designed for efficient range queries
- Key is split into:
  - Attribute name
  - Attribute value
- Example:
  - Server 1 $\rightarrow$ Price [1–50]
  - Server 2 $\rightarrow$ Price [51–100]
- **Trade-offs:**
  - Efficient queries
  - High update cost
  - Possible load imbalance

Conceptual Integration

- Flat naming → Identifier resolution.
- Structured naming → Path resolution.
- Attribute-based naming → Subspace search.
  - Entity = Point in N-dimensional space.
  - Query = Region in that space.
- Scalability requires:
  - Hierarchy (LDAP)
  - Distributed indexing
  - Space-filling curves
  - Hybrid designs
- Search is fundamentally harder than lookup.

Table of Contents

1. Names and Entities
2. Flat Naming
3. Structured Naming
4. Attribute-Based Naming
   - Distributed Indexing
5. Summary

Summary

- Names refer to entities in distributed systems.
- Three fundamental types of names:
  - Address: name of an access point.
  - Identifier: globally unique, stable, never reused.
  - Human-friendly name: readable character string.
- Flat Naming:
  - Identifiers must be resolved to addresses.
  - Core problem: efficient entity location.
  - Resolution techniques:
    - Broadcasting / multicasting
    - Forwarding pointers
    - Home-based tracking
    - Structured P2P systems (DHTs)
    - Hierarchical location services

Summary

- Structured Naming:
  - Names organized as a naming graph.
  - Entities referenced by path names.
  - Name resolution = traversal of the graph.
  - Large systems distribute nodes across multiple name servers.
- Attribute-Based Naming:
  - Entities described by (attribute, value) pairs.
  - Queries specify constraints over attributes.
  - Search replaces direct lookup.
- Scalability techniques:
  - Hierarchical directory services (LDAP)
  - Distributed indexing
  - DHT-based mapping
  - Space-filling curves for load balancing
- Core Insight:
  - Lookup is simple.
  - Search is fundamentally harder.